

16 February 2026

Introduction to Vector Databases: Embedding, Similarity, and Indexing

William Hanlon¹

1. Deakin University

Keywords

data, general topics for engineers, indexing, information retrieval, similarity, vector embeddings, vectors

Introduction to Vector Databases: Embedding, Similarity, and Indexing

William Hanlon
Deakin University, Australia

22 January 2026

Abstract

As digital technologies have grown quickly, about 90% of the world's data is now unstructured. Unlike traditional databases, unstructured data lacks standardised formats, making it hard to analyse. This paper examines how vector databases help solve this problem by enabling systems to find and use information based on meaning rather than just keywords. This paper explains how vector embeddings work, focusing on the Word2Vec model, which turns text into high-dimensional vectors. It also covers how cosine similarity measures how closely related texts are. Using a Python example with movie plot summaries, the paper shows how to turn text into vectors and search by meaning. It also discusses how these tools affect large language models through retrieval-augmented generation and examines the trade-offs between speed, complexity, and accuracy in approximate nearest neighbour search.

1 Background

Over the past decade, the amount of data we use and collect has grown rapidly, thanks to major advances in technology. Phones, for example, have changed from simple communication tools, to devices that can store and process large amounts of data. Online marketplaces and social media have also created huge amounts of unstructured data. Structured data is organised and follows strict rules, stored in relational databases with unique identifiers and links to other tables. In contrast, unstructured data does not follow a set rules or formats, making it difficult to prepare and analyse. Because of the growth in 'Big Data,' about 90% of all global data is now unstructured (Chen et al., 2014; Halevy et al., 2009; Coronel and Morris, 2023).

Unstructured data is hard to analyse without specialised skills because it lacks clear relationships or rules. One way to handle this is to convert the data to vectors and store them in a vector database. A vector is just a list of numbers that can be shown in many dimensions. For example, a vector for an animal like a mouse could be [1.001, 2.112, 4.0004, 2.9987], where each value corresponds

to a feature. This helps give the data meaning and link it to other points. A vector for a computer mouse, which could be represented as [7.223, 6.334, 3.121, 2.872], would be far from the animal mouse vector, indicating they aren't similar in meaning (Mikolov et al., 2013; Malkov and Yashunin, 2018; Taipalus, 2023).

Large Language Models (LLMs) utilise vectors to interpret meaning and generate relevant responses. These vectors function as a form of memory, enabling the LLM to retrieve relevant information. When a query is submitted, the LLM converts it into a high-dimensional vector, often comprising of thousands of dimensions. The database subsequently searches for vectors that are most similar to the query vector, a process known as semantic search (Brown et al., 2020; Lewis et al., 2020; Taipalus, 2023; Pan et al., 2023).

2 Vector Embeddings

A vector is simply an ordered list of numbers, called floats, which can be positive or negative and include decimals. But how do we turn a word like 'mouse' into a list of numbers, and how are those numbers decided? This is done through vector embeddings, which convert data like words into meaningful, multi-dimensional vectors (Mikolov et al., 2013; Jurafsky and Martin, 2023).

2.1 Word2Vec

There are several ways to create these embeddings, but here we will focus on Word2Vec. In 2013, Google published a paper called 'Efficient Estimation of Word Representation in Vector Space' that introduced two new models for representing words as vectors. The first is the Continuous Bag-of-Words (CBOW) model, which predicts a word based on the words around it (Mikolov et al., 2013).

Let's look at a fill-in-the-blank example. Take the sentence, "The hungry cat chased the mouse into the hole." To train the model, we pick a target word to hide, here it's 'mouse.' We also choose a window size; for this example, we'll use three words before and after the target. The model gets the words before ('cat', 'chased', 'the') and after ('into', 'the', 'hole'). So, the model sees "cat chased the ___ into the hole." It adds and averages the vectors for these words to guess the missing word. If it guesses 'dog,' it learns this is wrong and moves the 'dog' vector further away from the average. When it guesses 'mouse,' it is correct and moves the 'mouse' vector closer to the average of the context words (Mikolov et al., 2013; Jurafsky and Martin, 2023).

The Skip Gram model works the other way around. It predicts the words around a target word. Using the same sentence, 'mouse' is used to guess its neighbouring words. The model examines the vector for 'mouse' and identifies each nearby word. If it guesses wrong, it adjusts. If it gets words like 'cat' or 'hole' right, it strengthens the link between those words and 'mouse' (Mikolov et al., 2013; Jurafsky and Martin, 2023).

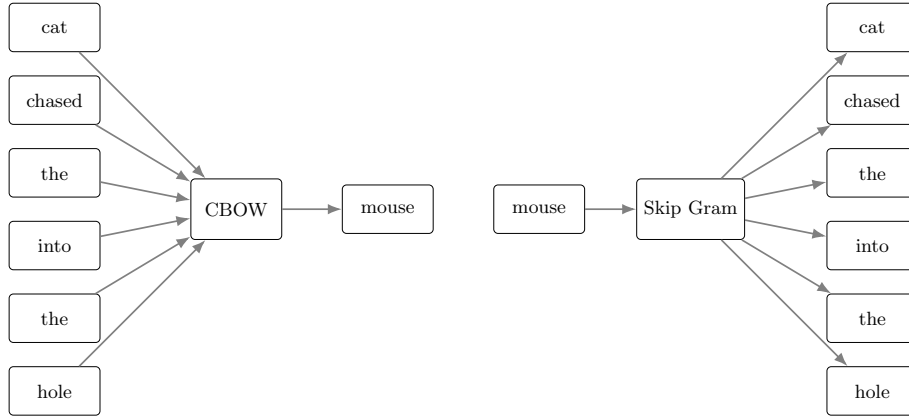


Figure 1: Architecture comparison: CBOW predicts the target word (‘mouse’) from context, while Skip-Gram uses the target word to predict the surrounding context.

3 Cosine Similarity

How can we tell if vectors are related? If we use the usual distance between two points, we ignore the length of the vectors. For example, a short sentence about a mouse and a long book about a mouse would have very different vector sizes, so they would seem unrelated even though they cover the same topic. Cosine similarity addresses this by measuring the angle between the vectors rather than the straight-line distance. It looks at the direction the vectors point (Manning et al., 2008; Jurafsky and Martin, 2023).

- **0° Angle:** If two vectors point in the exact same direction, the cosine is 1. This means the documents are perfectly similar in context, regardless of their length.
- **90° Angle:** If the vectors are perpendicular, the cosine is 0. This means the documents are unrelated (e.g., a “computer mouse” vs. a “living mouse”).
- **180° Angle:** If they point in opposite directions, the cosine is -1 , indicating opposite meanings (Manning et al., 2008; Jurafsky and Martin, 2023).

The similarity between two vectors can be calculated using Equation 1:

$$\text{similarity}(\mathbf{A}, \mathbf{B}) = \cos(\theta) = \frac{\mathbf{A} \cdot \mathbf{B}}{\|\mathbf{A}\| \|\mathbf{B}\|} = \frac{\sum_{i=1}^n A_i B_i}{\sqrt{\sum_{i=1}^n A_i^2} \sqrt{\sum_{i=1}^n B_i^2}} \quad (1)$$

If we measured the distance between the document and the query, the distance would be much larger, even though both are very similar topics (Manning et al., 2008).

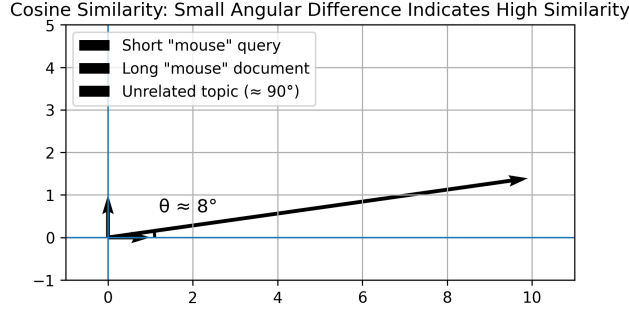


Figure 2: Visualisation of Cosine Similarity. The short query vector and the long document vector have a small angle ($\theta \approx 8^\circ$), indicating high similarity despite the difference in magnitude (length). The unrelated topic vector is perpendicular ($\theta = 90^\circ$).

By ignoring the length and focusing on the angle, we can see that in Figure 2 a short query and a long document are semantically similar because they point in the same direction in the vector space (Manning et al., 2008; Jurafsky and Martin, 2023).

4 Indexing

A real challenge appears when we try to use this method on a large scale. Calculating cosine similarity between two vectors is efficient, but real systems often have thousands, millions, or even billions of document embeddings. Comparing a query to every stored vector becomes slow and costly as the dataset grows (Manning et al., 2008; Malkov and Yashunin, 2018; Pan et al., 2023).

Embeddings and similarity scores help us measure relevance, but they don't solve the problem of doing so efficiently as the dataset grows. Without extra structure, the system would have to scan the entire database for every query, which doesn't scale well (Manning et al., 2008; Taipalus, 2023).

To solve this, vector databases use indexing. Vector indexing organises vectors so the system can quickly find the most relevant ones without having to check every vector. Just as traditional databases use indexes to speed up searches, vector indexes enable quick searches of large collections of high-dimensional data (Malkov and Yashunin, 2018; Taipalus, 2023).

The simplest way to search is Exact Search (also called k-Nearest Neighbour, or k-NN). This means comparing the query vector to every stored vector to find the best match. However, as the dataset grows, search time increases linearly, making it computationally intensive and too slow for real-world use cases (Manning et al., 2008).

To solve this, vector databases use Approximate Nearest Neighbour (ANN) indexing. Instead of scanning the entire database, ANN algorithms organise

vectors into efficient structures (such as graphs or clusters) that enable the system to quickly navigate to the most relevant neighbourhood. This trades a tiny, often negligible amount of accuracy for a massive increase in speed, enabling the system to search billions of vectors in milliseconds (Malkov and Yashunin, 2018; Taipalus, 2023; Pan et al., 2023).

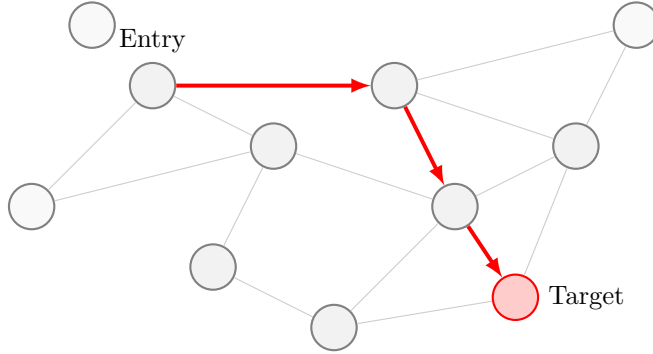


Figure 3: Visualising Index Search: Instead of checking every point (Linear Search), the algorithm navigates a ‘graph’ of connections, taking efficient ‘hops’ to find the Target quickly.

If we have a vector for an animal mouse and a vector for a computer mouse, the two are mapped to very different regions of vector space because their meanings were learned from different contexts. At a small scale, it is possible to compare a query vector against every stored vector to find which “mouse” is intended. However, in a real system with millions or billions of embeddings, this brute-force comparison is no longer practical. ANN indexing solves this by building structured neighbourhoods of meaning inside the vector space. Instead of treating each vector in isolation, ANN systems organise vectors into connected regions, where semantically similar items, such as animal or technology related words, are stored near one another. When a user searches for mouse, the system does not check every possible meaning. It begins at an entry point in the index and follows links through nearby vectors, moving step by step toward the part of the space that best matches the query. In this way, the algorithm quickly converges on the “computer mouse” or “animal mouse” region without scanning unrelated vectors, such as space galaxy or romantic movie. In modern systems, this navigation is implemented using Hierarchical Navigable Small World (HNSW) graphs, which organise vectors into multiple graph layers so that the search can make large jumps across the space at higher levels and then refine locally at lower levels, allowing the correct semantic neighbourhood to be found with high accuracy and very low latency. This layered graph structure is what enables ANN systems to dramatically reduce the number of distance calculations required while preserving high semantic accuracy (Malkov and Yashunin, 2018; Pan et al., 2023).

5 Example

Figure 4 shows a simple end-to-end vector search pipeline using unstructured text data. It starts with a small set of movie plot summaries. These summaries are split into words and used to train a Word2Vec model, which turns each word into a number-based vector based on its context. Since Word2Vec makes word-level vectors, each movie plot is turned into a single sentence vector by averaging its word vectors. These sentence vectors make up a basic vector database, with each entry holding a movie title, its vector, and the original text. When a user enters a query, it is embedded the same way and compared to every stored vector using cosine similarity. The results are ranked by similarity, and the most related document is returned. This method compares each vector to the query, which is easy to understand but not efficient for large datasets. Still, it clearly shows how embeddings, similarity, and retrieval work together for semantic search and to give context to a Large Language Model (Mikolov et al., 2013; Conneau et al., 2018; Manning et al., 2008; Malkov and Yashunin, 2018; Lewis et al., 2020; Řehůřek and Sojka, 2010; Pedregosa et al., 2011).

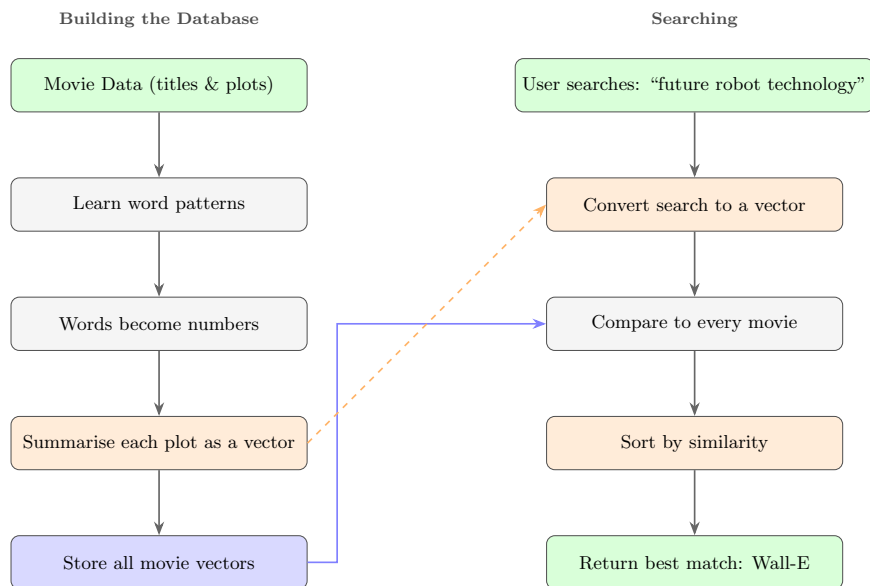


Figure 4: How the vector search works: First, the system learns from movie plots and stores each film as a list of numbers (a vector). When a user searches, their query is converted the same way and compared to every stored movie. The most similar result is returned.

6 Interpreting the Results

When a user enters the query “future robot technology,” the system turns it into a vector by averaging the vectors for “future,” “robot,” and “technology.” It then compares this query vector to each movie plot vector using cosine similarity. The search finds Wall-E as the best match, with a confidence score of 0.6262 (Mikolov et al., 2013; Conneau et al., 2018; Manning et al., 2008).

This result makes sense based on the movie plots. Wall-E shares several ideas with the query, like robot, space, and future, so its vector is close to the query vector. Let’s say Iron Man has the word technology in its movie plot, this doesn’t connect as strongly to space and future, so it gets a lower score. A cosine similarity of 0.6262 indicates a moderate to strong match; 1 means a perfect match, and 0 means no real connection (Manning et al., 2008; Jurafsky and Martin, 2023).

This finding illustrates how vector embeddings and cosine similarity allow the system to retrieve relevant information based on semantic meaning rather than exact keyword matches. Even though the query does not explicitly mention Wall-E or Earth, the shared contextual themes encoded in the vectors enable the model to identify it as the most relevant document. This retrieved context could then be passed to a Large Language Model to generate a coherent and informed response (Mikolov et al., 2013; Manning et al., 2008; Lewis et al., 2020).

6.1 Retrieval-Augmented Generation

When this text is retrieved and given to an LLM, it becomes part of the model’s input, not just a search result. The LLM reads the Wall-E plot along with the user’s question and uses it as a factual base for its answer. Instead of relying solely on its training data to guess what “future robot technology” means, the model uses a retrieved document that discusses robots, space, and a future Earth. This process, where semantic retrieval and language generation work together, is called Retrieval-Augmented Generation (RAG). Embeddings map both the user’s query and stored documents into the same high-dimensional vector space, allowing semantic similarity to be measured mathematically. The vector database then uses an ANN index to retrieve the most relevant text chunks based on this similarity, selecting the best contextual evidence for the query. These retrieved passages are injected into the prompt given to the Large Language Model, which uses them as grounding when generating a fluent, human-readable response (Lewis et al., 2020; Malkov and Yashunin, 2018; Zhao et al., 2023).

7 Opportunities / Benefits

7.1 Semantic Search

Semantic search in vector databases changes information retrieval from simple keyword matching to understanding meaning. This approach solves the “vocab-

ulary mismatch problem,” where traditional search fails if a user’s query does not match the exact words in a document. For example, searching for “leave policy” can find a document titled “Annual Time Off Guidelines” because their vector embeddings are similar. This feature automatically handles synonyms and words with multiple meanings, making it easier to find relevant unstructured data (Manning et al., 2008; Mikolov et al., 2013; Jurafsky and Martin, 2023).

7.2 Scalability

Indexing lets vector databases store and search billions of high-dimensional vectors efficiently, without needing to scan everything. Advanced indexing algorithms, like Hierarchical Navigable Small World (HNSW), build graph-based structures that help the system quickly find the right data in milliseconds. This scalability is important for platforms like streaming services, where recommendation engines need to quickly compare a user’s history to millions of movie vectors to give personalised content (Malkov and Yashunin, 2018; Taipalus, 2023; Pan et al., 2023).

7.3 RAG

Vector-based systems act as the “long-term memory” for Large Language Models (LLMs), supporting Retrieval-Augmented Generation (RAG). RAG lets organisations base AI responses on their own factual data, like legal rules or medical guidelines, instead of only using the LLM’s pre-trained knowledge. By finding relevant context before generating answers, organisations can greatly reduce errors and make sure clinicians or legal professionals get accurate, source-cited information (Lewis et al., 2020; Zhao et al., 2023).

7.4 Data Preparation

Traditional Relational Database Management Systems (RDBMS) need strict structures and lots of data cleaning before storing information. In contrast, vector databases can flexibly take in raw unstructured data, like text, logs, and audio transcripts. This makes data more accessible, allowing analysts to query different sources such as earnings reports or news feeds using natural language, and speeds up finding insights without complex SQL pipelines (Coronel and Morris, 2023; Taipalus, 2023).

8 Limitations / Challenges

8.1 Trade-off between accuracy and performance

To get search times under a second at scale, most vector databases use Approximate Nearest Neighbour (ANN) indexing instead of exact k-Nearest Neighbour

(k-NN) search. While ANNs are very efficient, they trade some recall (the ability to find every relevant item) for speed. This approach can sometimes miss the closest match, which is risky in areas like evidence discovery or healthcare, where precision is critical (Malkov and Yashunin, 2018; Taipalus, 2023; Pan et al., 2023).

8.2 Quality of Embeddings & Domain

The effectiveness of a vector database depends completely on the embedding model used. A big challenge is domain shift, where a general-purpose model (like one trained on Wikipedia) does not capture the details of specialised fields. For example, if a model does not know that “stroke” means different things in swimming and medicine, the vectors will be misaligned, leading to poor search results no matter how good the database is (Jurafsky and Martin, 2023; Taipalus, 2023; Zhao et al., 2023).

8.3 Infrastructure and Cost

Storing high-dimensional vectors (often 1,536 dimensions per entry) needs much more memory (RAM) than storing plain text. As datasets grow, the cost of calculating distances and keeping indexes in memory also rises. Organisations must balance the cost of special hardware, like GPUs, with the need for fast searches, often requiring complex engineering to handle the challenges of high-dimensional data (Malkov and Yashunin, 2018; Taipalus, 2023; Zhao et al., 2023; Pan et al., 2023).

8.4 Lack of Interpretability

Unlike keyword search, where matches are easy to explain (for example, “the document contains the word ‘apple’”), vector similarity is based on math that is hard to see. A document might be retrieved because it shares a hidden feature with the query that users cannot easily spot. This lack of explainability can reduce trust in industries where transparency and auditability are needed (Manning et al., 2008; Jurafsky and Martin, 2023; Ribeiro et al., 2016).

8.5 Data Freshness and Indexing

When documents are updated, the old vector must be removed and a new one added, which can require expensive restructuring of the index graph. This process can lead to a stale index, where the database does not show the most current information right away, making it harder to build real-time applications like stock news analysis (Malkov and Yashunin, 2018; Taipalus, 2023).

8.6 Semantic Closeness vs. Truth

In vector space, antonyms often appear close together because they are used in similar contexts (for example, “hot” and “cold” are both temperature words).

This means a semantic search might find a document that is the opposite of what the user wants, just because it is contextually relevant. Without extra filtering, this can cause Retrieval-Augmented Generation (RAG) systems to give answers based on conflicting or outdated information (Manning et al., 2008; Jurafsky and Martin, 2023; Lewis et al., 2020).

9 Conclusion

Moving from structured data management to vector-based semantic retrieval represents a major shift in how artificial intelligence processes human information. By turning unstructured data into multidimensional vector embeddings, organisations can go beyond the limitations of strict keyword searches and enable systems to understand context, intent, and nuance (Mikolov et al., 2013; Manning et al., 2008; Jurafsky and Martin, 2023).

This paper showed that with tools like Word2Vec and Cosine Similarity, differences in document length and complex phrasing can be handled to find semantically relevant connections, as seen in the Wall-E and Iron Man example. These abilities are the foundation of modern Retrieval-Augmented Generation (RAG) systems, which help Large Language Models use real, proprietary data and reduce errors (Mikolov et al., 2013; Manning et al., 2008; Lewis et al., 2020).

However, using vector databases comes with challenges, such as, the trade-off between search speed and precision, the high cost of high-dimensional indexing, and the difficulty of explaining why a particular result was retrieved. As unstructured data continues to grow, the future of AI will depend on improving these systems to balance scalability and interpretability. Vector databases have gone from a niche technology to a key part of the infrastructure that supports today’s intelligent systems (Malkov and Yashunin, 2018; Jurafsky and Martin, 2023; Taipalus, 2023; Pan et al., 2023).

References

- Conneau, A., Kiela, D., Schwenk, H., Barrault, L. and Bordes, A. (2018) Supervised learning of universal sentence representations from natural language inference data. *Proceedings of the 2017 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pp. 670–680. Available at: <https://arxiv.org/abs/1705.02364> (Accessed: 01 January 2026).
- Brown, T.B., Mann, B., Ryder, N., et al. (2020) Language models are few-shot learners. *Advances in Neural Information Processing Systems*, 33, pp. 1877–1901. Available at: <https://arxiv.org/abs/2005.14165> (Accessed: 01 January 2026).
- Chen, M., Mao, S. and Liu, Y. (2014) Big data: A survey. *Mobile Networks and Applications*, 19(2), pp. 171–209. Available at: <https://doi.org/10.1007/s11036-013-0489-0> (Accessed: 22 January 2026).

- Coronel, C. and Morris, S. (2023) *Database Systems: Design, Implementation, & Management*. 14th edn. Boston: Cengage Learning.
- Halevy, A., Norvig, P. and Pereira, F. (2009) The unreasonable effectiveness of data. *IEEE Intelligent Systems*, 24(2), pp. 8–12. Available at: <https://doi.org/10.1109/MIS.2009.36> (Accessed: 05 January 2026).
- Jurafsky, D. and Martin, J.H. (2023) *Speech and Language Processing*. 3rd edn. (Draft). Stanford University. Available at: <https://web.stanford.edu/~jurafsky/slp3/> (Accessed: 03 January 2026).
- Lewis, P., Perez, E., Piktus, A., et al. (2020) Retrieval-augmented generation for knowledge-intensive NLP tasks. *Advances in Neural Information Processing Systems*, 33. Available at: <https://arxiv.org/abs/2005.11401> (Accessed: 07 January 2026).
- Malkov, Y.A. and Yashunin, D.A. (2018) Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 42(4), pp. 824–836. Available at: <https://arxiv.org/abs/1603.09320> (Accessed: 02 January 2026).
- Manning, C.D., Raghavan, P. and Schütze, H. (2008) *Introduction to Information Retrieval*. Cambridge: Cambridge University Press. Available at: <https://nlp.stanford.edu/IR-book/> (Accessed: 02 January 2026).
- Mikolov, T., Chen, K., Corrado, G. and Dean, J. (2013) Efficient estimation of word representations in vector space. *arXiv preprint arXiv:1301.3781*. Available at: <https://arxiv.org/abs/1301.3781> (Accessed: 05 January 2026).
- Pan, J.J., Wang, J. and Li, G. (2023) Survey of vector database management systems. *arXiv preprint arXiv:2310.14021*. Available at: <https://arxiv.org/abs/2310.14021> (Accessed: 03 January 2026).
- Pedregosa, F., Varoquaux, G., Gramfort, A., et al. (2011) Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, pp. 2825–2830. Available at: <https://jmlr.org/papers/v12/pedregosa11a.html> (Accessed: 06 January 2026).
- Ribeiro, M.T., Singh, S. and Guestrin, C. (2016) “Why should I trust you?”: Explaining the predictions of any classifier. *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pp. 1135–1144. Available at: <https://arxiv.org/abs/1602.04938> (Accessed: 08 January 2026).
- Řehůřek, R. and Sojka, P. (2010) Software framework for topic modelling with large corpora. *Proceedings of the LREC 2010 Workshop on New Challenges for NLP Frameworks*. Available at: <https://radimrehurek.com/gensim/> (Accessed: 02 January 2026).

Taipalus, T. (2023) Vector database management systems: Fundamental concepts, use-cases, and current challenges. *arXiv preprint arXiv:2309.11322*. Available at: <https://arxiv.org/abs/2309.11322> (Accessed: 01 January 2026).

Zhao, W.X., Zhou, K., Li, J., Tang, T., Wang, X., Hou, Y., Min, Y., et al. (2023) A survey of large language models. *arXiv preprint arXiv:2303.18223*. Available at: <https://arxiv.org/abs/2303.18223> (Accessed: 01 January 2026).

Appendix A: Python Code

The following Python code implements the vector search pipeline described in this paper. It uses Gensim for Word2Vec training and scikit-learn for cosine similarity calculation.

```
1 import numpy as np
2 from gensim.models import Word2Vec
3 from sklearn.metrics.pairwise import cosine_similarity
4
5 # 1. The Data (Unstructured Text)
6 raw_data = [
7     {"title": "Star Wars", "plot": "space galaxy war empire rebels force"},
8     {"title": "The Notebook", "plot": "love romance heart relationship couple"},
9     {"title": "Iron Man", "plot": "technology suit robot billionaire action"},
10    {"title": "Wall-E", "plot": "robot space future earth love"},
11 ]
12
13 # Prepare data for Word2Vec
14 training_data = [d['plot'].split() for d in raw_data]
15
16 # 2. Train Word2Vec model
17 model = Word2Vec(
18     sentences=training_data,
19     vector_size=10,
20     min_count=1,
21     window=5
22 )
23
24 print(f"Vector for 'space': {model.wv['space']}")
25
26 # 3. Sentence Embeddings
27 def get_sentence_vector(sentence):
28     words = sentence.lower().split()
```

```

29     word_vectors = [model.wv[w] for w in words if w in model
30                     .wv]
31
32     if not word_vectors:
33         return np.zeros(10)
34
35     return np.mean(word_vectors, axis=0)
36
37 # 4. Build vector database
38 vector_db = []
39
40 for movie in raw_data:
41     vec = get_sentence_vector(movie['plot'])
42     vector_db.append({
43         "title": movie['title'],
44         "vector": vec,
45         "plot": movie['plot']
46     })
47
48 # 5. Search function using cosine similarity
49 def search_movies(user_query):
50     query_vec = get_sentence_vector(user_query)
51     results = []
52
53     for entry in vector_db:
54         score = cosine_similarity(
55             [query_vec],
56             [entry['vector']]
57         )[0][0]
58         results.append((score, entry))
59
60     results.sort(key=lambda x: x[0], reverse=True)
61     return results[0]
62
63 # 6. Example query
64 query = "future robot technology"
65 best_score, best_match = search_movies(query)
66
67 print("--- SEARCH RESULTS ---")
68 print(f"User Query: '{query}'")
69 print(f"Best Match: {best_match['title']}")
70 print(f"Confidence: {best_score:.4f}")
71 print(f"Context: {best_match['plot']}")

```

Example Output

```

1 Vector for 'space': [-0.00536227  0.00236431  0.0510335
2                     0.09009273 -0.0930295

```

```
2  -0.07116809  0.06458873  0.08972988 -0.05015428
   -0.03763372]
3
4  --- SEARCH RESULTS ---
5  User Query: 'future robot technology'
6  Best Match: Wall-E
7  Confidence: 0.6262
8  Context: robot space future earth love
```